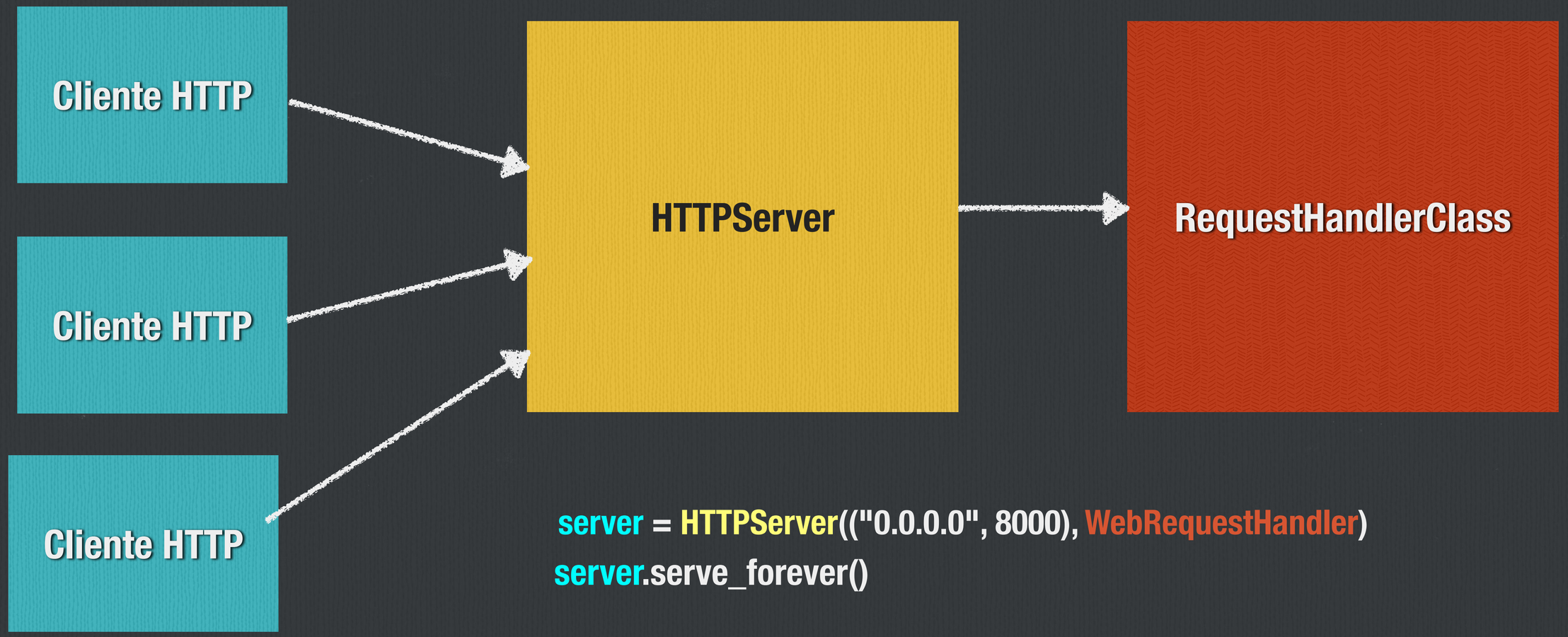
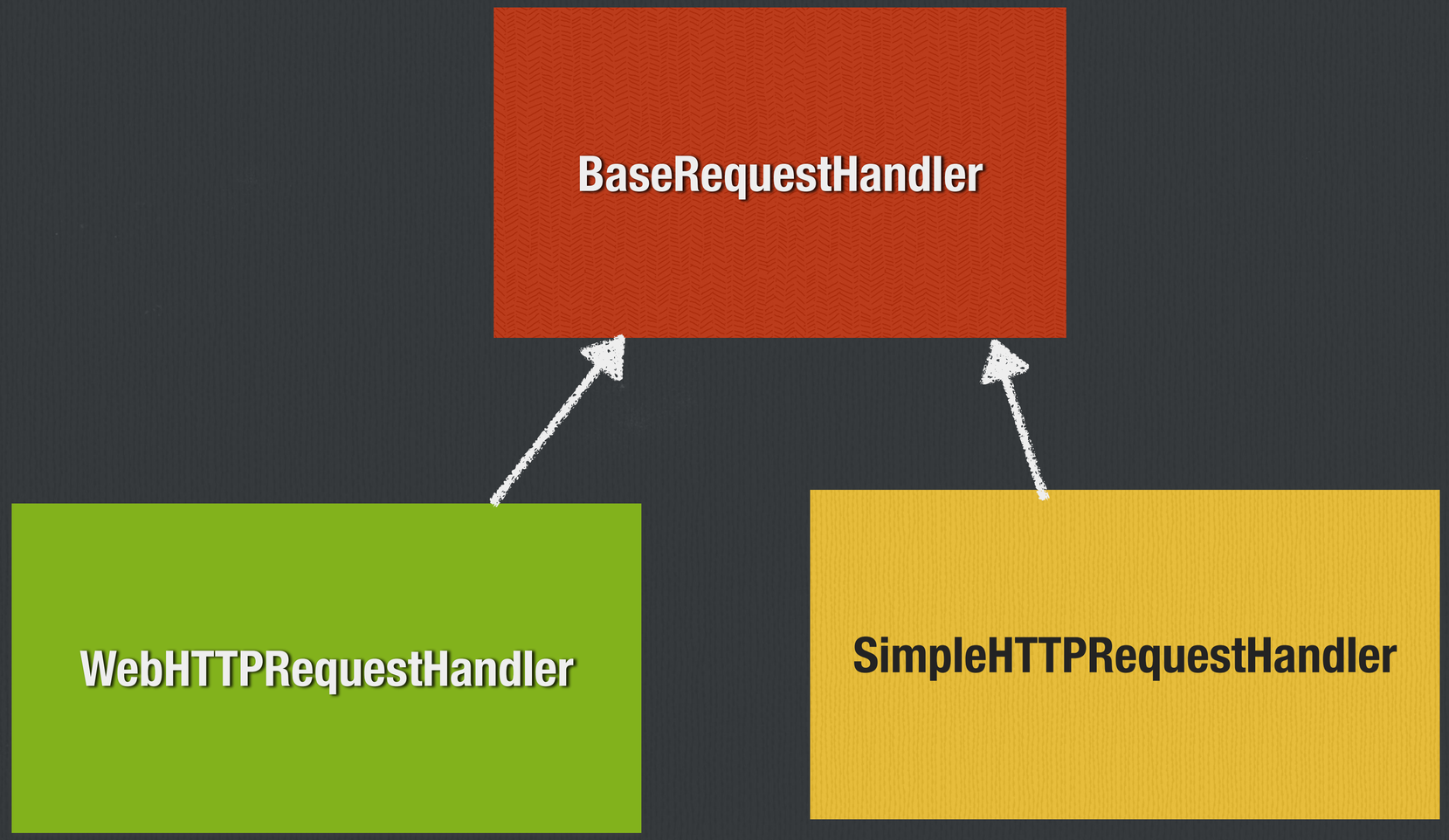
Servidor HTTP en Python

Componentes Principales

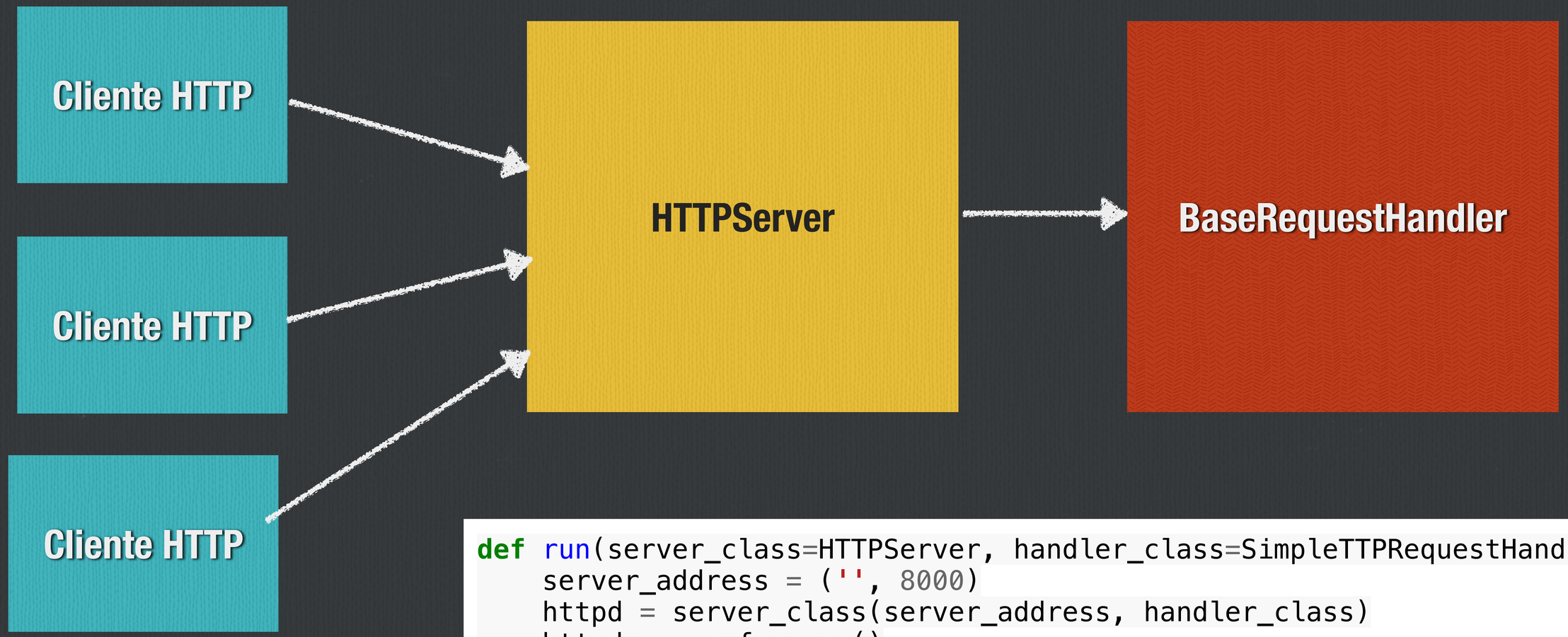



```
server = HTTPServer(("0.0.0.0", 8000), WebRequestHandler)
server.serve_forever()
```

RequestHandler



Componentes Principales




```python
def run(server_class=HTTPServer, handler_class=SimpleTTPRequestHandler):
    server_address = ('', 8000)
    httpd = server_class(server_address, handler_class)
    httpd.serve_forever()
```

class `http.server.`**`BaseHTTPRequestHandler`**`(`*`request, client_address, server`*`)`

Esta clase se utiliza para controlar las solicitudes HTTP que llegan al servidor. Por sí mismo, no puede responder a ninguna solicitud HTTP real; debemos implementar una subclase para manejar cada método de solicitud (por ejemplo, GET o POST).

`BaseHTTPRequestHandler` proporciona una serie de variables de clase e instancia, y métodos para su uso por subclases.

https://docs.python.org/es/3/library/http.server.html#http.server.BaseHTTPRequestHandler

- ☐ server
- ☐ command
- ☐ path
- ☐ headers
- ☐ rfile y wfile

- ☐ do_GET( )
- ☐ do_POST( )
- ☐ do_HEAD( )

- ☐ send_response(code, message=None)
- ☐ send_header(keyword, value)
- ☐ send_error(code, message=None, explain=None)
- ☐ end_headers( )

WebRequestHandler

webserver.py (1)

```python
from http.server import BaseHTTPRequestHandler, HTTPServer
from urllib.parse import parse_qsl, urlparse

class WebRequestHandler(BaseHTTPRequestHandler):
   def url(self):
        return urlparse(self.path)

   def query_data(self):
        return dict(parse_qsl(self.url.query))


   def do_GET(self):
        self.send_response(200)
        self.send_header("Content-Type", "text/html")
        self.end_headers()
        self.wfile.write(self.get_response().encode("utf-8"))

    def get_response(self):
        return f"""
    <h1> Hola Web </h1>
    <p> Path: {self.path}          </p>
    <p> Headers: {self.headers}       </p>
    <p> Query: {self.query_data}    </p>
"""
```

webserver.py (2)

```python
if __name__ == "__main__":
    print("Server starting...")
    server = HTTPServer(("0.0.0.0", 8000), WebRequestHandler)
    server.serve_forever()
```

urlparse

```
>>> from urllib.parse import urlparse
>>> urlparse("scheme://netloc/path;parameters?query#fragment")
ParseResult(scheme='scheme', netloc='netloc', path='/path;parameters', params='',
            query='query', fragment='fragment')
>>> o = urlparse("http://docs.python.org:80/3/library/urllib.parse.html?"
...              "highlight=params#url-parsing")
>>> o
ParseResult(scheme='http', netloc='docs.python.org:80',
            path='/3/library/urllib.parse.html', params='',
            query='highlight=params', fragment='url-parsing')
>>> o.scheme
'http'
>>> o.netloc
'docs.python.org:80'
>>> o.hostname
'docs.python.org'
>>> o.port
80
>>> o._replace(fragment="").geturl()
'http://docs.python.org:80/3/library/urllib.parse.html?highlight=params'
```

ParseResult (Named Tuple)

Attribute	Index	Value	Value if not present
`scheme`	0	URL scheme specifier	*scheme* parameter
`netloc`	1	Network location part	empty string
`path`	2	Hierarchical path	empty string
`params`	3	Parameters for last path element	empty string
`query`	4	Query component	empty string
`fragment`	5	Fragment identifier	empty string
`username`		User name	None
`password`		Password	None
`hostname`		Host name (lower case)	None
`port`		Port number as integer, if present	None

WebRequestHandler

webserver.py (1)

```python
from http.server import BaseHTTPRequestHandler, HTTPServer
from urllib.parse import parse_qsl, urlparse

class WebRequestHandler(BaseHTTPRequestHandler):
   def url(self):
        return urlparse(self.path)

   def query_data(self):
        return dict(parse_qsl(self.url.query))


   def do_GET(self):
        self.send_response(200)
        self.send_header("Content-Type", "text/html")
        self.end_headers()
        self.wfile.write(self.get_response().encode("utf-8"))

    def get_response(self):
        return f"""
    <h1> Hola Web </h1>
    <p> Path: {self.path}          </p>
    <p> Headers: {self.headers}       </p>
    <p> Query: {self.query_data}    </p>
"""
```

webserver.py (2)

```python
if __name__ == "__main__":
    print("Server starting...")
    server = HTTPServer(("0.0.0.0", 8000), WebRequestHandler)
    server.serve_forever()
```

curl://

- curl http://localhost:8080/ruta/otra\?qwer\=asdf\&otro\=qwer
- curl - v (verbose)
- POST (https://curl.se/docs/tutorial.html)
- User Agent
- Cookies